

What We Are Doing — A Plain-Language Guide to Our BTP

Project: Cross-Dataset Generalization of ML-Based Intrusion Detection Systems **For:** Shivam, Deepesh, Shantanu — so you can understand and defend the project **Read time:** ~15 minutes

This document explains the whole project in simple words, with analogies. If you understand this page, you can answer most viva questions. The companion file 08_viva_qa_prep.md has ready-made question-and-answer practice.

1. The one-paragraph summary (memorise this)

Companies use software called an **Intrusion Detection System (IDS)** to catch hackers on their network. Today, these systems are often built with **machine learning (ML)** — we show the computer lots of examples of “normal traffic” and “attack traffic”, and it learns to tell them apart. The problem is that researchers almost always test these systems on the **same dataset** they trained on, and get 99% accuracy. But when you take that system and put it on a **different company’s network**, it often fails badly — sometimes it is no better than flipping a coin. **Our project studies why this happens and tries to fix it** by choosing the *right features* using **bio-inspired optimisation algorithms**, so the model works across different networks — not just one.

2. Key ideas, explained simply

2.1 What is an IDS?

Think of a security guard watching CCTV footage of a building. An IDS is that guard for a computer network: it watches network traffic and raises an alarm when it sees something suspicious (a “malicious” or “attack” flow). There are two flavours: - **Signature-based:** a list of known “criminal faces”. Good at known attacks, blind to new ones. - **Anomaly/ML-based:** learns what *normal* looks like and flags anything unusual. This is what we use.

2.2 What is a “flow” and a “feature”?

Network traffic is a stream of packets. Packets that belong to the same conversation (same source/destination) form a **flow**. Instead of storing every packet, we summarise each flow with numbers called **features** — e.g. how long it lasted, how many bytes and packets went each way, the protocol used. So a dataset is a big table: **one row = one flow, columns = features, plus a label** saying whether it was an attack (and which kind).

2.3 What is “cross-dataset generalisation”?

- **Same-dataset test (what everyone does):** Train on 80% of *dataset A*, test on the other 20% of *dataset A*. → accuracy ~99%.

- **Cross-dataset test (what we do):** Train on *dataset A*, test on *dataset B* (a completely different network). → accuracy often collapses.

Analogy: It is the difference between a student practising *exactly the questions that will appear on the exam* (same-dataset), versus applying general knowledge to a totally new exam (cross-dataset). Our models are great at the first and bad at the second.

Generalisation = the ability to work on data you have *never seen before*. Cross-dataset generalisation = working on a *different network's* data.

2.4 Why does cross-dataset testing fail?

Three main reasons: 1. **Different features.** Each dataset was built with a different tool, so the columns do not match. A model that learned “column 12 is important” cannot even find column 12 in another dataset. 2. **Dataset-specific shortcuts (leakage).** Some features accidentally reveal *which dataset* or *which attack scenario* it is — e.g. IP addresses, port numbers, timestamps. The model “cheats” by memorising these, which works on the same dataset but fails on a new one. (Researchers call this a **shortcut** or **leakage**.) 3. **Different attack distributions.** The kinds of attacks, and how common each is, differ between networks.

2.5 What is “feature selection”?

A dataset may have 78 features, but many are redundant or misleading. **Feature selection** = automatically choosing a smaller subset of the most useful features. Benefits: faster, less overfitting, and — important for us — if we pick features that are *meaningful everywhere*, the model transfers better.

2.6 What is “metaheuristic / bio-inspired optimisation”?

Some problems are too large to check every possible answer. For 20 features there are about 1,000,000 possible subsets — too many to try all. **Metaheuristic algorithms** are smart search methods inspired by nature that find good-enough answers without checking everything: - **Genetic Algorithm (GA):** inspired by evolution. Start with many random feature-subsets, keep the best, “breed” and “mutate” them over generations. - **Particle Swarm Optimization (PSO):** inspired by flocks of birds. Many candidate solutions “fly” through the search space, sharing information to converge on good regions. - **Differential Evolution / Whale Optimization:** other nature-inspired searchers.

2.7 What is our key innovation?

Everyone who uses GA/PSO for IDS measures success by accuracy on the *same* dataset. We change the **fitness function** (the score the algorithm tries to maximise) to measure **cross-dataset performance** instead. In plain words:

Previous work asks the optimiser: “which features make the model accurate on *this* dataset?”

We ask: “which features make the model accurate on a *different* dataset too?”

Same tools (GA/PSO), different goal (transfer, not memorisation). That reframing is our contribution.

3. The datasets (know these)

Dataset	Who/what made it	Rows	Features	Flavours of attack
CIC-IDS-2017	Canadian Institute for Cybersecurity, with a flow tool called CICFlowMeter	2,830,743	78	DoS, DDoS, PortScan, Brute-force, Bot, Web attacks, etc.
UNSW-NB15	UNSW Canberra, using IXIA generator + Argus/Bro	257,673	42	DoS, Exploits, Fuzzers, Reconnaissance, Backdoor, Worms, etc.
TON_IoT	UNSW IoT testbed, using Zeek (Bro) logs	211,043	42	DDoS, DoS, Scanning, Ransomware, Injection, XSS, MITM, etc.

Why these three? They are the most-used modern benchmarks, and they are *structurally different* (different networks, different extraction tools) — exactly what a generalisation study needs.

What we already found: the three datasets have **no column name in common**. Only about 6 raw features mean the same thing in all three. That mismatch *is* the problem we study.

4. What we have done, and what is left

Done

- Downloaded and inspected all three datasets.
- Wrote scripts that compute real statistics (row counts, class distributions, missing/infinite values) and generate the charts.
- Reviewed 15 core research papers and identified the gap.
- Defined the objectives, the 7-phase methodology, and the evaluation protocol.
- Set up a reproducible code repository.

Left to do

1. Clean the data (fix infinite/NaN values, remove dangerous identity features, fix labels).
2. Harmonise the three datasets onto a **common feature set**.

3. Run the **baseline benchmark**: train on one dataset, test on another (the 3×3 matrix) — this shows how bad the gap is.
 4. Implement **GA and PSO with our cross-dataset fitness** and let them pick features.
 5. Compare against standard feature-selection baselines.
 6. Analyse which features/attacks transfer; write guidelines and the final report.
-

5. Likely “simple” questions and short answers

- “**So what is your project in one line?**” → We study why ML intrusion detectors fail on new networks and use bio-inspired optimisation to pick features that work across networks.
 - “**What is new?**” → Metaheuristic feature selection has never been aimed at *cross-dataset* performance before.
 - “**Why not just use all features?**” → Dataset-specific features mislead the model and cause the very failure we are trying to fix.
 - “**Won’t training on many datasets solve it?**” → Combining datasets still leaves mismatched columns; our shared-subspace approach handles that explicitly. (We aim only to *improve* the gap, not to make it zero.)
-

6. Glossary (quick reference)

Term	Simple meaning
Flow	One network “conversation”, summarised as numbers
Feature	A numeric column describing a flow
Label	The answer (benign / attack type)
IDS / NIDS	Intrusion Detection System (network version)
Generalisation	Working correctly on unseen data
Cross-dataset generalisation	Working on a <i>different</i> dataset/network
Overfitting	Memorising training data instead of learning general rules
Leakage / shortcut	Features that reveal the answer artificially (e.g. IPs, ports)
Feature selection	Choosing a useful subset of features
Metaheuristic	Nature-inspired search (GA, PSO, ...)
Fitness function	The score an optimiser tries to maximise
Class imbalance	Some classes have far more examples than others
Balanced accuracy / macro-F1	Metrics that treat all classes fairly (used because of imbalance)
Domain adaptation	Techniques to align two datasets; an alternative approach (needs target data)